

GLA UNIVERSITY, GREATER NOIDA

Greater Noida – 201310, Uttar Pradesh, India

FlightPulse: An AI-Powered Flight Delay Prediction and Route Risk Analytics System

SOFTWARE REQUIREMENT SPECIFICATION

Version 1.0

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING - AI

Submitted By

Vidushi Saraswat

Roll No.: 22515500100 | Section: NE

Under the Supervision of

Palash Raghuvanshi

GLA University, Greater Noida

SCHOOL / DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING / APPLICATIONS

GLA UNIVERSITY, GREATER NOIDA – 201310

September, 2026

Revision History

Version	Date	Author	Description
v1.0	25-Sep-2026	Vidushi Saraswat	Initial SRS — project changed from disease-surge prediction to flight delay prediction (FlightPulse)

1. Introduction

1.1 Purpose

This document specifies the software requirements for FlightPulse, a full-stack application that predicts the probability and expected severity of a flight delay for a given route, using historical flight data. It is intended to guide development and to give evaluators a falsifiable list of what the system will and will not do in this release.

1.2 Scope

The system ingests historical flight records (airline, origin, destination, scheduled times, distance, and outcome), fits a statistical and machine learning model to that data, and serves predictions through a web dashboard. A user enters a route and flight details and receives a delay probability, an expected delay in minutes, and a plain risk level (Low/Medium/High), along with the factors behind that risk. The system does not book flights, does not use live/real-time flight-tracking data, and is trained on historical United States flight data in this release — a large, free Indian flight-delay dataset was not available at the time of writing, so the system is not presented as India-specific.

1.3 Definitions

Term	Definition
Delay probability	The model's estimated chance, expressed as a percentage, that a given flight will depart or arrive later than scheduled.
Expected delay	The predicted length of the delay, in minutes, if a delay occurs.
Route risk	A simple Low/Medium/High label summarising historical delay performance for a specific origin–destination pair.

2. Overall Description

2.1 User Classes

User Class	Description	Technical Level
General Traveller	Enters a route and flight details to check delay risk before booking or travelling.	Non-technical
Student/Developer	Uploads new data, retrains and evaluates the model.	Technical

2.2 Assumptions and Dependencies

- A sufficiently large, free, historical flight-delay dataset (United States flight data, approximately 5.82 million records) remains accessible for download.
- Historical performance for a route/airline is a reasonable indicator of future delay risk, even though it cannot account for one-off events (e.g. a single day's severe weather).
- The Python ML microservice and Java backend can communicate over local HTTP during development.

2.3 Constraints

C1 The system shall use only publicly available historical flight data; it shall not connect to any live flight-tracking or airline booking system.

C2 The system shall be described as trained on United States historical flight data; it shall not be presented as an India-specific predictor unless trained on Indian data in a later version.

3. Data Source and Baseline

Dataset identity, license, and structure are confirmed below. Row count, missing values, and the exact date range are left for after the file is actually downloaded and opened — those numbers should never be guessed.

Field	Value
Confirmed dataset	Kaggle — "2015 Flight Delays and Cancellations" (U.S. DOT Bureau of Transportation Statistics), kaggle.com/datasets/usdot/flight-delays, CC0 Public Domain. Three files: flights.csv (main data), airlines.csv, airports.csv.
Confirmed fields	flights.csv has 31 columns, including airline, origin airport, destination airport, scheduled departure time, scheduled arrival time, distance, and delay/cancellation outcome fields.
Row count	Publicly documented as "more than 5.8 million rows" by multiple independent sources; exact figure to be confirmed with .shape after loading — not stated precisely anywhere, so not guessed here.
Missing values	Not yet known — to confirm with .isnull().sum() after loading. Diverted and cancelled flights are expected to have missing delay values by nature, not by error.
Date range	Covers calendar year 2015 (1 Jan – 31 Dec 2015), confirmed from the dataset's own publisher description.
Known limitation	Data is U.S.-only; results describe U.S. route/airline patterns and should not be generalised to Indian aviation without separate validation

4. System Architecture

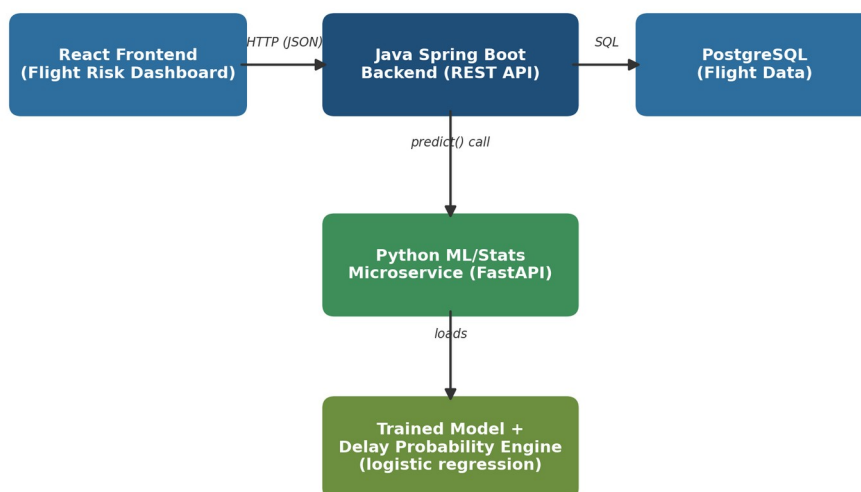


Figure 1: System architecture — React frontend, Java Spring Boot backend, Python ML/stats microservice, PostgreSQL database

Component	Responsibility
React Frontend	Lets the user enter flight/route details; displays delay probability, expected delay, risk level, and supporting charts; calls the Java backend over HTTP

Java Spring Boot Backend	Exposes REST endpoints, manages PostgreSQL, calls the Python microservice for predictions
Python Microservice	Cleans data, computes statistics (mean/variance/correlation), trains/serves the ML model
PostgreSQL	Stores cleaned flight data and a simple record of each prediction next to the real outcome, once known

5. Functional Requirements

5.1 Data Upload and Validation

FR-1.1 The system shall accept a CSV file of historical flight records via an upload endpoint.

FR-1.2 The system shall reject a CSV file missing required columns (airline, origin, destination, scheduled time, delay outcome), returning an error naming the missing column.

FR-1.3 The system shall replace missing numeric values using median imputation and record how many values were imputed.

FR-1.4 The system shall store the cleaned dataset in PostgreSQL, replacing the previous version for that data source.

5.2 Model-backed Requirements

FR-2.1 The system shall accept a request specifying airline, origin, destination, and scheduled date/time, and reject a request missing any of these with a 400 error.

FR-2.2 The system shall return a delay probability (percentage) and an expected delay (in minutes) for a valid request.

FR-2.3 The system shall present a Low/Medium/High route risk label together with the top factors contributing to that risk.

FR-2.4 (stretch, optional) The system shall, when the Python microservice is unavailable, return the most recent stored prediction with a timestamp instead of failing silently.

FR-2.5 (stretch, optional) The system shall allow a Student/Developer user to trigger a model retrain using the currently stored dataset.

6. Non-Functional Requirements

NFR-1 The system shall return a prediction result within 3 seconds of a valid /predict request under normal load.

NFR-2 The system shall require a fixed key value in the request header for the /upload and /retrain endpoints, rejecting requests without it.

NFR-3 The system shall log every prediction request with a timestamp and a unique request ID in PostgreSQL.

NFR-4 The system shall remain usable by a non-technical user without documentation, verified by having one person outside the project complete a route risk lookup unassisted.

7. Build Plan

7.1 Minimum Viable Product

The MVP is: FR-1.1, FR-1.2, FR-1.3, FR-2.1, FR-2.2, FR-2.3.

The MVP is the smallest version of the project that still works start to finish — upload flight data, clean it, and get back a real delay probability and risk level for a chosen route. These six requirements form one complete, working path through the system; nothing else is useful until this path exists.

FR-2.4 and FR-2.5 are left out of the MVP on purpose — they add convenience (fallback behaviour, retraining on demand) once the core path already works, but are not needed for the project to be considered complete and working.

7.2 Build Order

Target completion: December 2026. Stages below are spaced to leave a buffer before the final deadline.

Stage	Target Date	Requirements Targeted
Data collection & cleaning	10-Oct-2026	FR-1.1, FR-1.2, FR-1.3, FR-1.4
Statistics & probability layer	25-Oct-2026	FR-2.1, FR-2.2, FR-2.3
ML model & backend integration	15-Nov-2026	FR-2.2, FR-2.4
Frontend & dashboard	30-Nov-2026	FR-2.2, FR-2.3 (display)
Retrain workflow & polish	10-Dec-2026	FR-2.5, NFR-1–NFR-4

8. Data Requirements

8.1 Entities

Entity: FlightRecord

Field	Type	Notes
airline	text	Carrier code or name
origin	text	Origin airport code
destination	text	Destination airport code
scheduled_departure	datetime	Planned departure time
distance	integer	Route distance
delay_minutes	integer	Actual delay; 0 or negative if on-time/early

Other entities: PredictionLog (stores each prediction and, once known, the real outcome, so the two can be compared); User (Student/Developer login for upload and retrain access).

8.2 Entity Relationship Diagram

NOT REQUIRED FOR v1.0 — to be completed in a later version.

9. Design Decisions and Alternatives Considered

The reasoning for each choice is kept short enough to say out loud and defend without notes.

Decision	Alternative Considered	Reason for Choice (simple)
Split the backend into two parts: Java handles the app and database; Python handles the maths/ML	Write the whole backend in just one language (only Python)	The project needs to show both Java and Python. Splitting the work this way lets each language do the part it is naturally better at.
Use logistic regression to predict delay probability	Use a more complex model (e.g. deep learning) from the start	Logistic regression is a standard, well-understood method for yes/no-style predictions like "delayed or not", and is realistic to learn, build, and explain within a

		second-year timeline.
Use U.S. historical flight data instead of waiting for an Indian dataset	Delay the whole project until a large enough Indian flight-delay dataset is found	No sufficiently large, free Indian flight-delay dataset was found. The project is described honestly as U.S.-data-based rather than mislabelled as India-specific.

10. Acceptance Criteria

AC-1 Given a valid CSV upload, when submitted via the /upload endpoint, the system stores the cleaned data and returns a success response listing the number of rows imputed.

AC-2 Given a valid /predict request for a specified route and time, the system returns a delay probability, expected delay, and risk level within 3 seconds.

AC-3 Given the Python microservice is stopped, when a /predict request is made, the system returns the last stored prediction with its timestamp instead of a server error.

AC-4 Given a CSV upload missing the "delay_minutes" column, when submitted, the system rejects it and returns an error naming that column.

11. Out of Scope for This Release

Real-time flight tracking or live delay updates are not part of this release — the system works from historical data only, since a live flight-tracking API is not something accessible at the student level.

This release does not attempt an India-specific version. A large, free Indian flight-delay dataset could not be found during planning, so the system is built and described honestly as U.S.-data-based rather than mislabelled.

Flight booking, rebooking, and alerting are out of scope — this is a risk/probability predictor, not a booking or notification platform, and leaving these out keeps v1.0 achievable.

Appendix A: State Model

NOT REQUIRED FOR v1.0 — to be completed in a later version.

Appendix B: Traceability

NOT REQUIRED FOR v1.0 — to be completed once task-board and test-case IDs exist.